

BC28 KeyChange UpgradeCodeunit

BC28: Tabellen-Key ändern ohne Datenverlust via Upgrade- Codeunit

Business Central 28 (2025 Release Wave 1)
Stand: 29.06.2026

Problemstellung

In AL kann eine Tabelle nach der ersten Veröffentlichung nicht destruktiv geändert werden. Ein Key-Wechsel oder -Hinzufügen ist destruktiv und führt zu Datenverlust, wenn man es direkt macht.

Lösung: Tabelle duplizieren + Upgrade- Codeunit mit UpgradeTags

Prinzip

1. **Alte Tabelle** → ObsoleteState = Pending + Umbenennen
 2. **Neue Tabelle** mit korrigiertem Schlüssel anlegen
 3. **Upgrade-Codeunit** mit UpgradeTags (BC28!) schreibt Daten rüber
 4. Start-NAVAppDataUpgrade führt die Tags aus
-

Schritt 1: Alte Tabelle obsolet setzen

```
table 50100 "MyTable Obsolete"
{
    ObsoleteState = Pending;
    ObsoleteReason = 'Migrated to MyTable with new key structure';

    fields
    {
        field(1; "Entry No."; Integer) { }
        field(2; "Old Key Field"; Code[20]) { }
        field(3; Description; Text[100]) { }
    }

    keys
    {
        key(PK; "Entry No.") { Clustered = true; }
        key(OldKey; "Old Key Field") { } //    dieser Key soll
geändert werden
    }
}
```

Schritt 2: Neue Tabelle mit korrigiertem Key

```
table 50101 "MyTable"
{
    DataClassification = ToBeClassified;

    fields
    {
        field(1; "Entry No."; Integer) { DataClassification =
ToBeClassified; }
        field(2; "New Key Field"; Code[20]) { DataClassification =
ToBeClassified; }
        field(3; Description; Text[100]) { DataClassification =
ToBeClassified; }
    }

    keys
    {
        key(PK; "Entry No.") { Clustered = true; }
        key(ChangedKey; "New Key Field") { } //    neuer /
geänderter Key
    }
}
```

```
}  
}
```

Schritt 3: BC28 Upgrade-Codeunit mit UpgradeTags

```
codeunit 50100 "MyTable Key Migration"  
{  
    Subtype = Upgrade;  
    TableNo = 50101;           //      neue  
Tabelle  
    UpgradeTags = 'MyTable-NewKey-v1.1';           //      BC28:  
eindeutiger Tag  
  
    trigger OnUpgradePerDatabase()  
    begin  
        // Schema-Änderungen auf DB-Ebene (1x, nicht pro Mandant)  
    end;  
  
    trigger OnUpgradePerCompany()  
    begin  
        MigrateDataToNewTable();  
    end;  
  
    local procedure MigrateDataToNewTable()  
    var  
        OldRec: Record "MyTable Obsolete";  
        NewRec: Record MyTable;  
    begin  
        if OldRec.IsEmpty() then  
            exit;  
  
        OldRec.Reset();  
        if OldRec.FindSet() then  
            repeat  
                NewRec.Init();  
                NewRec."Entry No." := OldRec."Entry No.";  
                NewRec."New Key Field" := OldRec."Old Key Field";  
                NewRec.Description := OldRec.Description;  
                NewRec.Insert();  
            until OldRec.Next() = 0;  
  
        // Nur leeren, NACHDEM die Kopie erfolgreich war
```

```
        OldRec.DeleteAll();
    end;
}
```

BC28 vs. ältere Versionen

Feature	Vor BC28	BC28
Identifikation	Nur Subtype = Upgrade + TableNo	Zusätzlich UpgradeTags
Ausführungs-Tracking	Läuft bei jedem Versionssprung	Tag-basiert → läuft nur 1× pro Tag
Mehrere Upgrades	1 Codeunit pro Tabelle	Beliebig viele mit verschiedenen Tags
DB-Ebene	Nur OnUpgradePerCompany	+ OnUpgradePerDatabase

Schritt 4: Deployment & Data Upgrade

App veröffentlichen

```
Publish-NAVApp -ServerInstance BC -Path ".\MyApp_1.1.0.0.app"
-SkipVerification
```

Schema synchronisieren

```
Sync-NAVTenant -ServerInstance BC -Force
```

Data Upgrade starten (führt alle neuen UpgradeTags aus)

```
Start-NAVAppDataUpgrade -ServerInstance BC -Name "MyApp" -Version
"1.1.0.0"
```

BC28 trackt automatisch, welche Tags schon ausgeführt wurden.

MyTable-NewKey-v1.1 wird **nie doppelt** ausgeführt.

Späteres Schema-Update: weitere Codeunit mit neuem Tag

```
codeunit 50101 "MyTable Index Rebuild"
{
    Subtype = Upgrade;
```

```

TableNo = 50101;
UpgradeTags = 'MyTable-IndexRebuild-v1.2';    // anderer Tag

trigger OnUpgradePerDatabase()
begin
    // Indizes neu aufbauen o.Ä.
end;
}

```

Entscheidungsmatrix: Welches Vorgehen für welchen Key?

Key-Änderung	Vorgehen
Primärschlüssel ändern	Tabelle umbenennen + neue Tabelle + Upgrade-Codeunit
Sekundärschlüssel hinzufügen/ändern	Gleiches Vorgehen (kein Schema-Update möglich)
Nur Key-Eigenschaft (z.B. Clustered)	Optional per Codeunit, nicht immer nötig
Key löschen	Neue Tabelle ohne den Key

Häufige Fehler & Lösungen

Fehler	Lösung
Start-NAVAppDataUpgrade läuft nicht	Vorher Sync-NAVTenant -Force ausführen
„Table already exists“	Alte und neue Tabelle unterschiedlich benennen
Datenverlust	DeleteAll() NUR nach erfolgreichem Kopieren
ObsoleteState = Removed	Nie direkt auf Removed setzen → erst Pending
UpgradeCodeunit läuft bei jedem Publish	UpgradeTags setzen → BC28 cached den Tag

Upgrade Codeunit Execution Flow (BC28)

